Connect a Foundry IQ knowledge base to Foundry Agent Service

> **Important**
>
> Features, capabilities, or properties marked (preview) aren't covered by a service-level agreement, aren't recommended for production workloads, and might change or be constrained before they become generally available. The Azure AI Search preview terms apply to all preview functionality, whether it's standalone or part of a generally available feature.

In this article, you learn how to connect a knowledge base in Foundry IQ to an agent in Foundry Agent Service. The connection uses the Model Context Protocol (MCP) to facilitate tool calls. When invoked by the agent, the knowledge base orchestrates retrieval across its configured knowledge sources. Depending on its configuration, the knowledge base can:

- Use LLM-based query planning (preview) to decompose a user query into subqueries.
- Process the subqueries simultaneously by using keyword, vector, or hybrid search.
- Apply semantic reranking to identify the most relevant results.
- Use answer synthesis (preview) to combine the results into a natural-language response with source references.

The agent uses the retrieved or synthesized response to ground its answers in enterprise data or web sources. Source references help users verify the information and trace it to the originating content.

For an end-to-end example of integrating Azure AI Search and Foundry Agent Service for knowledge retrieval, see the agentic-retrieval-pipeline-example Python sample on GitHub.

Usage support

Expand table

Microsoft Foundry support	Python SDK	C# SDK	JavaScript SDK	Java SDK	REST API	Basic agent setup	Standard agent setup
✔	✔	-	-	-	✔	✔	✔

Prerequisites

- An Azure AI Search service with a knowledge base containing one or more knowledge sources.
- A Microsoft Foundry project with an LLM deployment, such as `gpt-4.1-mini`. Hub-based projects aren't supported.
- Authentication and permissions on your search service and project.
- The latest preview Python SDK (version 2.0.0 or later) or the 2026-08-01-preview REST API version.

```bash
pip install "azure-ai-projects>=2.0.0" requests
```

Authentication and permissions

We recommend role-based access control for production deployments. To assign the roles in this section, you need the **Owner** or **User Access Administrator** role on both resources, or another role that grants `Microsoft.Authorization/roleAssignments/write`. If roles aren't feasible, skip this section and use key-based authentication instead.

Microsoft Foundry

- On the parent resource of your project, you need the **Foundry User** role to access model deployments and create agents. **Owners** automatically get this role when they create the resource. Other users need a specific role assignment. For more information, see Role-based access control in Foundry portal.

  > **Important**
  >
  > The Foundry RBAC roles were recently renamed. **Foundry User**, **Foundry Owner**, **Foundry Account Owner**, and **Foundry Project Manager** were previously named Azure AI User, Azure AI Owner, Azure AI Account Owner, and Azure AI Project Manager. You might still see the previous names in some places while the rename rolls out. The role IDs and core permissions are unchanged by the rename.

- On the parent resource of your project, you need the **Foundry Project Manager** role to create a project connection for MCP authentication and either **Foundry User** or **Foundry Project Manager** to use the MCP tool in agents.
- (Conditional) On the parent resource of your project, assign the **Cognitive Services User** role to your search service's system-assigned managed identity. This step is required only if your knowledge base specifies an LLM. Depending on its configuration, the knowledge base uses this identity to call the LLM for query planning, answer synthesis, or both. For more information, see Connect to Azure AI Search using a managed identity.
- On your project, create a system-assigned managed identity for interactions with Azure AI Search.

Required values

Use the following values in the code samples.

Expand table

Value	Where to get it	Example
Project endpoint (`project_endpoint`)	Find it in your project details in the Microsoft Foundry portal.	`https://your-resource.services.ai.azure.com/api/projects/your-proje`
Project resource ID (`project_resource_id`)	Copy the project ARM resource ID from Azure portal or use Azure CLI to query the resource ID. Your Microsoft Foundry project must have the `Microsoft.CognitiveServices/accounts` namespace.	`/subscriptions/.../resourceGroups/.../providers/Microsoft.Cognitive`
Azure AI Search endpoint (`search_service_endpoint`)	Find it on your Azure AI Search service **Overview** page (the service URL) in the Azure portal.	`https://your-search-service.search.windows.net`
Knowledge base name (`knowledge_base_name`)	Use the knowledge base name you created in Azure AI Search.	`hr-policy-kb`
Project connection name (`project_connection_name`)	Choose a name for the project connection you create.	`my-kb-mcp-connection`

Value	Where to get it	Example
Agent name (`agent_name`)	Choose a name for the agent version you create.	`hr-assistant`
Model deployment name (`deployed_LLM`)	Find it in your Microsoft Foundry project model deployments.	`gpt-4.1-mini`

> **Tip**
>
> We recommend you store the project endpoint, search endpoint, and knowledge base name in a `.env` file for local development.

Create a project connection

Create a `RemoteTool` connection (preview) on your Microsoft Foundry project. This connection uses the project's managed identity to target the MCP endpoint of the knowledge base, allowing the agent to securely communicate with Azure AI Search for retrieval operations.

> **Note**
>
> The `RemoteTool` category and `ProjectManagedIdentity` authentication type are specific to Microsoft Foundry project connections.

Python

Python

```python
import requests
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

# Provide connection details
credential = DefaultAzureCredential()
project_resource_id = "{project_resource_id}" # e.g.
/subscriptions/{subscription}/resourceGroups/{resource_group}/providers/Microsoft
.CognitiveServices/accounts/{account_name}/projects/{project_name}
project_connection_name = "{project_connection_name}"
mcp_endpoint = "
{search_service_endpoint}/knowledgebases/{knowledge_base_name}/mcp?api-
version=2026-08-01-preview" # This endpoint enables the MCP connection between
the agent and knowledge base

# Get bearer token for authentication
bearer_token_provider = get_bearer_token_provider(credential,
"https://management.azure.com/.default")
headers = {
  "Authorization": f"Bearer {bearer_token_provider()}",
}

# Create project connection
response = requests.put(

f"https://management.azure.com{project_resource_id}/connections/{project_connecti
on_name}?api-version=2025-10-01-preview",
  headers = headers,
  json = {
    "name": project_connection_name,
    "type": "Microsoft.MachineLearningServices/workspaces/connections",
    "properties": {
      "authType": "ProjectManagedIdentity",
      "category": "RemoteTool",
      "target": mcp_endpoint,
      "isSharedToAll": True,
      "audience": "https://search.azure.com/",
      "metadata": { "ApiType": "Azure" }
    }
  }
)
```

```
response.raise_for_status()
print(f"Connection '{project_connection_name}' created or updated successfully.")
```

Optimize agent instructions for knowledge retrieval

To improve knowledge base invocations and produce citation-backed answers, start with instructions like the following:

plaintext

```
You are a helpful assistant.

Use the knowledge base tool to answer user questions.
If the knowledge base doesn't contain the answer, respond with "I don't know".

When you use information from the knowledge base, include citations to the retrieved
sources.
```

This instruction template optimizes for:

- **Higher MCP tool invocation rates**: Explicit directives ensure the agent consistently calls the knowledge base tool rather than relying on its training data.
- **Clear source attribution**: Citations make it easier to validate where information came from.

> **Tip**
>
> While this template provides a strong foundation, evaluate and iterate on the instructions based on your specific use case and objectives. Test different variations to find what works best for your scenario.

Create an agent with the MCP tool

> **Warning**
>
> MCP implementations are susceptible to risks, such as attacks, cascading failures, and loss of human oversight. You can mitigate these risks by vetting MCP servers for security and reliability, following Microsoft's recommended practices and industry best practices, and implementing approval mechanisms and monitoring cascading behaviors.

Create an agent that integrates the knowledge base as an MCP tool. The agent uses a system prompt to instruct when and how to call the knowledge base. It follows instructions on how to answer questions and automatically maintains its tool configuration and settings across conversation sessions.

Add the knowledge base MCP tool with the project connection you previously created. This tool orchestrates query planning, decomposition, and retrieval across configured knowledge sources. The agent uses this tool to answer queries.

> **Note**
>
> Azure AI Search knowledge bases expose the `knowledge_base_retrieve` MCP tool for agent integration. This is the only tool currently supported for use with Foundry Agent Service.

Python

Python

```python
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition, MCPTool
from azure.identity import DefaultAzureCredential

# Provide agent configuration details
credential = DefaultAzureCredential()
mcp_endpoint = "
{search_service_endpoint}/knowledgebases/{knowledge_base_name}/mcp?api-
version=2026-08-01-preview"
project_endpoint = "{project_endpoint}" # e.g. https://your-foundry-
resource.services.ai.azure.com/api/projects/your-foundry-project
project_connection_name = "{project_connection_name}"
agent_name = "{agent_name}"
agent_model = "{deployed_LLM}" # e.g. gpt-4.1-mini

# Create project client
project_client = AIProjectClient(endpoint = project_endpoint, credential = cre-
dential)

# Define agent instructions (see "Optimize agent instructions" section for guid-
ance)
instructions = """
You are a helpful assistant that must use the knowledge base to answer all the
questions from user. You must never answer from your own knowledge under any cir-
cumstances.
Every answer must always provide annotations for using the MCP knowledge base
tool and render them as: `【message_idx:search_idx†source_name】`
If you cannot find the answer in the provided knowledge base you must respond
with "I don't know".
"""

# Create MCP tool with knowledge base connection
mcp_kb_tool = MCPTool(
    server_label = "knowledge-base",
    server_url = mcp_endpoint,
    require_approval = "never",
    allowed_tools = ["knowledge_base_retrieve"],
    project_connection_id = project_connection_name
)

# Create agent with MCP tool
agent = project_client.agents.create_version(
    agent_name = agent_name,
    definition = PromptAgentDefinition(
        model = agent_model,
        instructions = instructions,
        tools = [mcp_kb_tool]
    )
)

print(f"Agent '{agent_name}' created or updated successfully.")
```

Create the agent with the .NET SDK

Install the prerelease packages with `dotnet add package Azure.AI.Projects --prerelease` and `dotnet add package Azure.Identity`.

C#

```csharp
using Azure.AI.Projects;
using Azure.AI.Projects.Agents;
using Azure.Identity;
using OpenAI.Responses;

#pragma warning disable AAIP001, OPENAI001

var mcpEndpoint = "{search_service_endpoint}/knowledgebases/{knowledge_base_name}/mcp?
api-version=2026-05-01-preview";
var projectEndpoint = "{project_endpoint}"; // e.g. https://your-foundry-
resource.services.ai.azure.com/api/projects/your-foundry-project
var projectConnectionName = "{project_connection_name}";
var agentName = "{agent_name}";
var agentModel = "gpt-4.1-mini";

AIProjectClient projectClient = new(new Uri(projectEndpoint), new
DefaultAzureCredential());
var agentsClient = projectClient.AgentAdministrationClient;
```

```
// Define agent instructions (see "Optimize agent instructions" section for guidance).
var instructions = """
    You are a helpful assistant that must use the knowledge base to answer all the
questions from user. You must never answer from your own knowledge under any circum-
stances.
    Every answer must always provide annotations for using the MCP knowledge base tool
and render them as: `【message_idx:search_idx†source_name】`
    If you cannot find the answer in the provided knowledge base you must respond with
"I don't know".
    """;

// Create an MCP tool that points at the knowledge base connection.
McpTool mcpKbTool = ResponseTool.CreateMcpTool(
    serverLabel: "knowledge-base",
    serverUri: new Uri(mcpEndpoint),
    allowedTools: new McpToolFilter { ToolNames = { "knowledge_base_retrieve" } },
    toolCallApprovalPolicy: new
McpToolCallApprovalPolicy(GlobalMcpToolCallApprovalPolicy.NeverRequireApproval));
mcpKbTool.ProjectConnectionId = projectConnectionName;

// Create the agent with the MCP tool.
DeclarativeAgentDefinition definition = new(model: agentModel)
{
    Instructions = instructions,
    Tools = { mcpKbTool },
};
ProjectsAgentVersion agent = agentsClient.CreateAgentVersion(
    agentName: agentName,
    options: new ProjectsAgentVersionCreationOptions(definition));
Console.WriteLine($"Agent '{agentName}' created or updated successfully.");
```

(Optional) Enforce permissions with per-request headers

If any of your knowledge sources contain permission-protected content, the retrieval engine can filter results so that each user sees only the documents they're authorized to access. Choose how to set the `x-ms-query-source-authorization` header:

- **Set dynamically by Agent Service**: When the calling principal is a user, Agent Service sets the header from that user's context. This behavior applies whether the incoming request targets Foundry or the agent blueprint.
- **Set explicitly by the application**: Supply a user's Azure AI Search token through structured inputs when the calling principal doesn't directly represent that user or when the application needs to select the user context for each request.

The following pattern demonstrates the explicit option. Without the token, permission-enabled sources return results unfiltered. For more information, see Enforce permissions at query time (preview).

To vary MCP headers per request, such as passing a different user's token on each call, declare a structured input in the agent definition and reference it as a `{{placeholder}}` in the tool's `headers`. The caller supplies the value on each invocation. This approach works for MCP tools bound to a project connection.

For per-user authorization against an MCP server, you can also use OAuth identity passthrough.

Python

Update the agent from the previous step so the MCP tool reads its authorization header from a structured input:

Python

```
from azure.ai.projects.models import StructuredInputDefinition

# Reference the token as a placeholder in the header
mcp_kb_tool = MCPTool(
    server_label = "knowledge-base",
    server_url = mcp_endpoint,
    require_approval = "never",
```

```
    allowed_tools = ["knowledge_base_retrieve"],
    project_connection_id = project_connection_name,
    headers = {
        "x-ms-query-source-authorization": "{{search_auth_token}}"
    }
)

# Declare the structured input so the caller can supply the token per request
agent = project_client.agents.create_version(
    agent_name = agent_name,
    definition = PromptAgentDefinition(
        model = agent_model,
        instructions = instructions,
        tools = [mcp_kb_tool],
        structured_inputs = {
            "search_auth_token": StructuredInputDefinition(
                description = "Per-user Azure AI Search bearer token",
                required = True,
                schema = {"type": "string"},
            )
        }
    )
)

print(f"Agent '{agent_name}' created or updated successfully.")
```

When you invoke the agent, supply an Azure AI Search token in `structured_inputs`. This example resolves a token from the current `credential`. For a multi-user app, pass the token of each signed-in user instead. For example, use a token obtained through an on-behalf-of flow so the retrieval engine can filter results for that user.

Python

```
# Resolve an Azure AI Search token from the current credential (use a per-user
token in production)
from azure.identity import get_bearer_token_provider

search_token = get_bearer_token_provider(credential,
"https://search.azure.com/.default")()

openai_client = project_client.get_openai_client(agent_name=agent.name)
conversation = openai_client.conversations.create()

response = openai_client.responses.create(
    conversation = conversation.id,
    input = "{user_query}",
    extra_body = {
        "structured_inputs": {"search_auth_token": search_token},
    },
)
```

Invoke the agent with a query

Create a conversation session and send a user query to the agent. When appropriate, the agent orchestrates calls to the MCP tool to retrieve relevant content from the knowledge base. The agent then synthesizes this content into a natural-language response that cites the source documents.

Citation URLs in agent responses vary by knowledge source. For example, blob knowledge sources return the original document URL, while search index knowledge sources fall back to the MCP endpoint of your knowledge base.

Python

Python

```
# Get the OpenAI client for responses and conversations
openai_client = project_client.get_openai_client(agent_name=agent.name)

# Create conversation
conversation = openai_client.conversations.create()
```

```
# Send request to trigger the MCP tool
response = openai_client.responses.create(
    conversation = conversation.id,
    input = """
        Why do suburban belts display larger December brightening than urban
cores even though absolute light levels are higher downtown?
        Why is the Phoenix nighttime street grid is so sharply visible from
space, whereas large stretches of the interstate between midwestern cities remain
comparatively dim?
    """,
)

print(f"Response: {response.output_text}")
```

The output should be similar to the following (truncated for brevity):

```
Response: Suburban belts display larger December brightening than urban cores,
even
though absolute light levels are higher downtown, primarily because holiday
lights
increase most dramatically in the suburbs and outskirts of major cities. This is
due
to more yard space and a prevalence of single-family homes in suburban areas...

The Phoenix nighttime street grid is sharply visible from space due to the city's
layout along a regular grid of city blocks and streets with extensive street
lighting...

References:
- earth_at_night_508_page_174, earth_at_night_508_page_176 (Holiday lighting)
- earth_at_night_508_page_104, earth_at_night_508_page_105 (Phoenix grid
visibility)
```

Delete the agent and project connection

Python

Python

```
# Delete the agent
project_client.agents.delete_version(agent_name=agent.name,
agent_version=agent.version)
print(f"Agent '{agent.name}' version '{agent.version}' deleted successfully.")

# Delete the project connection (Azure Resource Manager)
import requests
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

credential = DefaultAzureCredential()
project_resource_id = "{project_resource_id}"
project_connection_name = "{project_connection_name}"

bearer_token_provider = get_bearer_token_provider(credential,
"https://management.azure.com/.default")
headers = {"Authorization": f"Bearer {bearer_token_provider()}"}

response = requests.delete(

f"https://management.azure.com{project_resource_id}/connections/{project_connecti
on_name}?api-version=2025-10-01-preview",
  headers=headers,
)
response.raise_for_status()
print(f"Project connection '{project_connection_name}' deleted successfully.")
```

> **Note**
>
> Deleting your agent and project connection doesn't delete your knowledge base or its knowledge sources. You must delete these objects separately on your Azure AI Search

service. For more information, see Delete a knowledge base and Delete a knowledge source.

Troubleshooting

This section helps you troubleshoot common issues when connecting Foundry Agent Service to a Foundry IQ knowledge base.

Authorization failures (401/403)

- If you get a 403 from Azure AI Search, confirm the project's managed identity has the **Search Index Data Reader** role on the search service (and **Search Index Data Contributor** if you write to indexes).
- If you get a 403 from Azure Resource Manager when you create or delete the project connection, confirm your user or service principal has permissions on the Microsoft Foundry resource and project.
- If you use keyless authentication, confirm your environment is signed in to the correct tenant and subscription.

MCP endpoint errors (400/404)

- Confirm `search_service_endpoint` is the Azure AI Search service URL, such as `https://<name>.search.windows.net`.
- Confirm `knowledge_base_name` matches the knowledge base you created in Azure AI Search.
- Confirm you use the `2026-08-01-preview` API version for the knowledge base MCP endpoint.

The agent doesn't ground answers

- Confirm the agent has the MCP tool configured and `allowed_tools` includes `knowledge_base_retrieve`.
- Update your agent instructions to explicitly require using the knowledge base and to return "I don't know" when retrieval doesn't contain the answer.

Related content

- Create a knowledge base in Azure AI Search
- Tutorial: Build an end-to-end agentic retrieval solution
- Foundry IQ: Unlocking ubiquitous knowledge for agents
- Tool best practices

Note: The author created this article with assistance from AI. Learn more

Last updated on 09/17/2026